

# Python基础语法与进阶语法(判断)

该文档内容是课程中的关键词/关键知识点的精简整理，同时给出了每个知识点详细的代码示例

文档中写好的代码可以直接复制尝试

请务必学习完对应课节，尝试完成课程作业之后，再来回顾此份讲义，否则你会觉得自己和计算机无缘

## 1. 字面量与数据类型

### 1.1. 字面量 (Literal)

字面量是指在程序中直接写出的固定值。这些值是“常量”，即在程序运行过程中不会改变的值。Python 支持多种类型的字面量，包括数字、字符串、布尔值等

字面量示例 Python

```
1 # 数字字面量
2 10
3 3.14
4
5 # 字符串字面量
6 "Hello World!"
7
8 # 布尔字面量
9 True
```

### 1.2. 数据类型 (Data Types)

数据类型是用于区分不同类型数据的分类方式。Python 提供了多种内置数据类型，每种类型都有其特定的属性和方法。了解并正确使用这些数据类型是编写高效、准确程序的基础。

Python 中的主要数据类型包括：

| 数据类型 | 描述 | 示例 |
|------|----|----|
|------|----|----|

|                         |             |                                         |
|-------------------------|-------------|-----------------------------------------|
| 整数 <code>int</code>     | 整数          | <code>10</code> , <code>-5</code>       |
| 浮点数 <code>float</code>  | 浮点数         | <code>3.14</code> , <code>-0.001</code> |
| 复数 <code>complex</code> | 复数          | <code>1 + 2j</code>                     |
| 字符串 <code>str</code>    | 字符串         | <code>"Hello, World!"</code>            |
| 数组 <code>list</code>    | 列表（有序可变集合）  | <code>1, 2, 3, 4</code>                 |
| 元组 <code>tuple</code>   | 元组（有序不可变集合） | <code>(1, 2, 3, 4)</code>               |
| 集合 <code>set</code>     | 集合（无序不重复元素） | <code>{1, 2, 3}</code>                  |
| 字典 <code>dict</code>    | 字典（键值对集合）   | <code>{"name": "Le", "age": 25}</code>  |
| 布尔类型 <code>bool</code>  | 布尔值         | <code>True</code> , <code>False</code>  |
| <code>NoneType</code>   | 表示空值或无值     | <code>None</code>                       |

### 1.3. 查看某个数值的类型

在 Python 中，可以使用内置函数 `type()` 和 `isinstance()` 来查看某个数值的类型

- **`type()`** 函数

用于返回对象的类型

#### ▼ `type()`函数示例

Python |

```

1  print(type(10))                # <class int>
2  print(type(3.14))              # <class float>
3  print(type("Hello, World!"))  # <class str>
4  print(type(1, 2, 3))           # <class list>
5  print(type((1, 2, 3)))         # <class tuple>
6  print(type({1, 2, 3}))         # <class set>
7  print(type({"name": "Alice"})) # <class dict>
8  print(type(True))              # <class bool>
9  print(type(None))              # <class NoneType>

```

- `instance()` 函数

函数用于判断一个对象是否是指定类型或其子类的实例。它返回 `True` 或 `False`

▼ `instance()`函数示例

Python |

```
1 print(isinstance(10, int))           # True
2 print(isinstance(3.14, float))       # True
3 print(isinstance("Hello, World!", str)) # True
4 print(isinstance(1, 2, 3, list))     # True
5 print(isinstance((1, 2, 3), tuple))  # True
6 print(isinstance({1, 2, 3}, set))    # True
7 print(isinstance({"name": "Alice"}, dict)) # True
8 print(isinstance(True, bool))       # True
9 print(isinstance(None, type(None)))  # True
10
11 # 也可以检查是否为多种类型之一
12 print(isinstance(10, (int, float))) # True
13 print(isinstance(3.14, (int, float))) # True
```

## 2. 变量与标识符

### 2.1. 变量

在 Python 中，变量是用于存储数据的命名位置。变量的值可以在程序执行过程中改变，且变量本身没有固定的类型（Python 是一种动态类型语言）

变量的定义格式： `变量名称 = 变量的值`

▼ 变量定义示例

Python |

```
1 # 定义变量并赋值
2 x = 10
3 name = "Alice"
4 price = 19.99
5 is_valid = True
6
7 # 变量的值可以随时改变
8 x = 20
9 name = "Bob"
```

## 2.2. 标识符

在 Python 中，标识符（identifier）是用于命名变量、函数、类、模块或其他对象的名称。选择合适的标识符名称有助于提高代码的可读性和可维护性

命名规则：

- 必须以字母（a–z, A–Z）或下划线（\_）开头
- 只允许出现英文字母、数字、下划线
- 对大小写敏感（例如，name 和 Name 是不同的变量）
- 不可使用 Python 保留关键字

常见的 Python 保留关键字：

```
False, await, else, import, pass, None, break, except, in, raise,  
True, class, finally, is, return, and, continue, for, lambda, try,  
as, def, from, nonlocal, while, assert, del, global, not, with,  
async, elif, if, or, yield
```

常用命名规范：

- 下划线命名法（snake\_case）：单词之间用下划线分隔，如 first\_name、total\_price
- 小驼峰命名法（camelCase）：第一个单词小写，后续单词首字母大写，如 firstName、totalPrice
- 大驼峰命名法（PascalCase）：每个单词首字母都大写，如 FirstName、TotalPrice

命名追求：

- 语义化：名称应能反映变量的用途或含义
- 清晰明了：名称应易于理解和记忆
- 减少长度：在保持可读性的前提下，尽量简短
- 尽量避免拼音：使用英文单词命名，以便国际化和团队协作

良好的命名示例Python

```
1 # 通过遵循这些命名规则和规范，可以使代码更加清晰易读，提高开发效率和代码质量
2 student_name = "Alice" # 下划线命名法
3 totalPrice = 99.99 # 小驼峰命名法
4 EmployeeID = 12345 # 大驼峰命名法
5
6 # 不推荐的命名示例
7 x = 10 # 变量名过于简短，缺乏语义
8 zongjia = 99.99 # 使用拼音命名，不利于理解
```

### 3. 运算符和表达式

在编程中，运算符（operator）是用于执行各种操作的符号。运算符可以用于数学运算、比较、逻辑运算、赋值、位运算等

#### 3.1. 赋值运算符 =

用于将值分配给变量

赋值运算符示例Python

```
1 x = 10
2 name = "Alice"
```

#### 3.2. 算术运算符（Arithmetic Operators）

算术运算符用于执行基本数学运算。以下是常见的算术运算符及其解释和示例：

| 运算符 | 解释 | 示例            |
|-----|----|---------------|
| +   | 加法 | 5 + 3 结果是 8   |
| -   | 减法 | 5 - 3 结果是 2   |
| *   | 乘法 | 5 * 3 结果是 15  |
| /   | 除法 | 5 / 2 结果是 2.5 |

|    |          |              |
|----|----------|--------------|
| // | 取整除（地板除） | 5 // 2 结果是 2 |
| %  | 取余数（模运算） | 5 % 2 结果是 1  |
| ** | 幂运算      | 2 ** 3 结果是 8 |

注意：加运算符 `+` 可以在多种数据类型之间操作

▼ +运算符示例

Python |

```
1 # 数值加法
2 result = 5 + 3 # 结果是 8
3
4 # 字符串拼接
5 greeting = "Hello, " + "World!" # 结果是 "Hello, World!"
6
7 # 列表合并
8 combined_list = [1, 2, 3] + [4, 5, 6] # 结果是 [1, 2, 3, 4, 5, 6]
```

3.3. 复合赋值运算符

复合赋值运算符将运算符与赋值操作结合在一起。以下是常见的复合赋值运算符及其解释和示例：

| 运算符              | 解释    | 示例                                                  |
|------------------|-------|-----------------------------------------------------|
| <code>+=</code>  | 加法赋值  | <code>c += b</code><br>等效于 <code>c = c + b</code>   |
| <code>-=</code>  | 减法赋值  | <code>c -= b</code><br>等效于 <code>c = c - b</code>   |
| <code>*=</code>  | 乘法赋值  | <code>c *= b</code><br>等效于 <code>c = c * b</code>   |
| <code>/=</code>  | 除法赋值  | <code>c /= b</code><br>等效于 <code>c = c / b</code>   |
| <code>//=</code> | 取整除赋值 | <code>c //= b</code><br>等效于 <code>c = c // b</code> |

|                  |       |                                                     |
|------------------|-------|-----------------------------------------------------|
| <code>%=</code>  | 取余数赋值 | <code>c %= b</code><br>等效于 <code>c = c % b</code>   |
| <code>**=</code> | 幂运算赋值 | <code>c **= b</code><br>等效于 <code>c = c ** b</code> |

## 3.4. 表达式

表达式是有明确执行结果的代码，它在被计算后会产生一个值。

▼ 表达式示例 Python |

```

1  # 算术表达式打印输出
2  print(5 + 3 * 2) # 结果是 11
3
4  # 布尔表达式
5  is_greater = 10 > 5 # 结果是 is_greater被赋值了True
6
7  # 函数调用表达式
8  length = len("Hello, World!") # 结果是 length被赋值了13

```

任何能放变量的地方，也能放表达式

## 4. 字符串精讲

### 4.1. 字符串的三种定义形式

在 Python 中，字符串可以使用单引号（'）、双引号（"）和三引号（''' 或 ""''）来定义。不同的定义方式在某些情况下具有特定的用途和优势

#### 4.1.1. 单引号（'）

用于定义短小的字符串，特别是当字符串中不包含单引号时。

```
1 single_quote_string = 'Hello, World!'
```

#### 4.1.2. 双引号 (")

用于定义包含单引号的字符串，避免转义字符的使用。

```
1 double_quote_string = "It's a beautiful day!"
```

#### 4.1.3. 三引号 ('' 或 ''')

用于定义多行字符串，或者包含单引号和双引号的字符串。也常用于多行注释。

```
1 triple_quote_string = """This is a
2 multi-line string."""
```

## 4.2. 字符串的拼接与格式化

字符串的拼接可以使用 `+` 运算符，但也有更优雅的方法，其中包括占位符 (%) 和 `f-string`。

#### 4.2.1. 占位符 (%)

占位符用于在字符串中插入变量。常用的占位符有 `%s`、`%d` 和 `%f`。

通俗解释：我先占个位置，一会儿会有变量过来，我把他变成字符串替换我的位置

| 占位符             | 解释  | 示例                                                                        |
|-----------------|-----|---------------------------------------------------------------------------|
| <code>%s</code> | 字符串 | <pre>"Hello, %s!" % "World"</pre> <p>结果是 <code>"Hello, World!"</code></p> |



|                 |     |                                                                                                 |
|-----------------|-----|-------------------------------------------------------------------------------------------------|
| <code>%d</code> | 整数  | <code>"There are %d apples" % 5</code><br>结果是 <code>"There are 5 apples"</code>                 |
| <code>%f</code> | 浮点数 | <code>"Pi is approximately %f" % 3.14</code><br>结果是 <code>"Pi is approximately 3.140000"</code> |

可以使用辅助符号 "m.n" 控制宽度和精度：

- `m` 控制宽度，要求是数字。如果设置的宽度小于数字自身宽度，不生效
- `.n` 控制小数点精度，要求是数字，会进行小数的四舍五入

m.n控制精度示例 Python |

```

1  # 控制浮点数精度
2  pi = 3.1415926535
3  print("Pi to 2 decimal places: %.2f" % pi) # 结果是 "Pi to 2 decimal place
   s: 3.14"
4
5  # 控制宽度
6  number = 42
7  print("Number with width 5: %5d" % number) # 结果是 "Number with width 5:
   42"

```

#### 4.2.2. 简便写法 `f-string`

Python3.6之后开始支持

`f-string` 是一种更简便的字符串格式化方式，它允许我们直接在字符串中嵌入表达式

例子：

f-string示例 Python |

```

1  # 使用 f-string 格式化字符串
2  name = "Bob"
3  height = 1.75
4  formatted_string = f"My name is {name} and my height is {height:.2f} meter
   s."
5  print(formatted_string) # 结果是 "My name is Bob and my height is 1.75 mete
   rs."

```

通过这些方法，可以灵活地对字符串进行拼接和格式化，以满足不同的需求

## 5. 缩进与注释

### 5.1. 代码缩进

在 Python 中，缩进（indentation）用于表示代码块的层次结构和逻辑关系。与许多其他编程语言不同，Python 不使用大括号 `{ }` 来定义代码块，而是依赖缩进来表示代码块的开始和结束。这使得代码更简洁和易读，但也要求严格的缩进规则

- 缩进可以使用空格或制表符（Tab），但在同一个代码块中只能使用一种方式
- 官方建议使用 4 个空格作为一个缩进层次
- 缩进的层次结构必须严格一致，不能混用不同数量的空格或制表符

#### ▼ 缩进示例1

Python |

```
1  # 正确的缩进
2  if True:
3      print("This is a correct indentation")
4  if True:
5      print("This is another correct indentation")
6
7  # 错误的缩进
8  if True:
9      print("This is a correct indentation")
10 if True: # 错误的缩进
11     print("This will cause an IndentationError")
```

相同缩进级别的语句属于同一个代码块

#### ▼ 缩进示例2

Python |

```
1 if True:
2     print("This is part of the first code block")
3     print("This is also part of the first code block")
4 print("This is outside the first code block")
```

### 5.2. 代码注释

在 Python 中，注释是代码中的非执行部分，用于解释代码、标注信息或提醒开发者注意事项。注释对于代码的可读性、可维护性和协作性非常重要

### 5.2.1. 单行注释

使用 `#` 符号开头。单行注释常用于对某一行代码或某一段代码进行简短的说明。

#### ▼ 单行注释示例

Python |

```
1  # 这是一个单行注释
2  x = 10 # 在代码后面的单行注释
```

### 5.2.2. 多行注释

有两种方式可以创建多行注释

#### 1. 使用连续的 `#` 符号：

#### ▼ 多行注释示例1

Python |

```
1  # 这是一个多行注释
2  # 这种方式使用多个单行注释
3  # 每一行都以 # 开头
```

#### 2. 使用三引号 (`'''` 或 `"""`)

这实际上是一个多行字符串，但可以用作注释。这种方式的注释在实际运行时会被解析为字符串，但不会执行，因此常用作注释

#### ▼ 多行注释示例2

Python |

```
1  """
2  这是一个多行注释
3  使用三引号 (''' 或 """) 包裹
4  这种方式也可以用于定义多行字符串
5  """
```

```
1 # 这是一个单行注释
2
3 """
4 这是一个多行注释
5 可以使用三引号包裹
6 这种方式在实际运行时会被解析为字符串
7 """
8
9 def example_function():
10     # 在函数内的单行注释
11     x = 10 # 变量赋值的注释
12     y = 20
13
14     # 计算和注释
15     result = x + y # 计算 x 和 y 的和
16     return result
```

## 6. 判断（条件）语句

在 Python 中，判断语句（也称为条件语句）用于根据不同条件执行不同的代码块

### 6.1. if 语句

`if` 语句用于在条件为真时执行代码块，语法如下

```
1 if 条件:
2     代码块
```

```
1 x = 10
2 if x > 5:
3     print("x is greater than 5")
```

### 6.2. if else 语句

**if else** 语句用于在条件为真时执行一个代码块，否则执行另一个代码块，语法如下

if else语法伪代码

Python |

```
1 if 条件:
2     代码块1
3 else:
4     代码块2
```

if else代码示例

Python |

```
1 x = 3
2 if x > 5:
3     print("x is greater than 5")
4 else:
5     print("x is not greater than 5")
```

## 6.3. if elif else 语句

**if else else** if elif else 语句用于在多个条件之间选择执行一个代码块，语法如下

if elif else语法伪代码

Python |

```
1 if 条件1:
2     代码块1
3 elif 条件2:
4     代码块2
5 else:
6     代码块3
```

if elif else示例

Python |

```
1 x = 7
2 if x > 10:
3     print("x is greater than 10")
4 elif x > 5:
5     print("x is greater than 5 but not greater than 10")
6 else:
7     print("x is 5 or less")
```

## 6.4. 嵌套使用

判断语句可以嵌套使用，以检查多个条件

```
1 x = 8
2 if x > 5:
3     print("x is greater than 5")
4     if x > 10:
5         print("x is also greater than 10")
6     elif x >= 8:
7         print("x is greater than 8 and lower than 10")
8     else:
9         print("x is greater than 5 and not greater than 8")
10 else:
11     print("x is not greater than 5")
```